

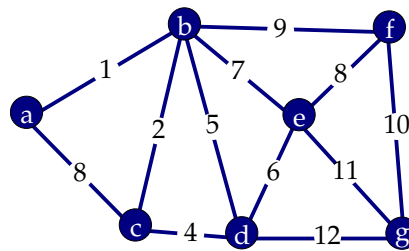
RO202 - Initiation à la Recherche Opérationnelle

Zacharie ALES, Tiphaine GEORGE
2025 - 2026

EXERCICES - Arbres couvrants et plus courts chemins

Exercice 1 Arbres

Une banque veut connecter ses 7 succursales via un réseau. Les coûts de construction des liens entre les succursales sont donnés sur le graphe ci-dessous. La banque ne dispose que d'un budget limité et a décidé de construire un réseau en arbre.



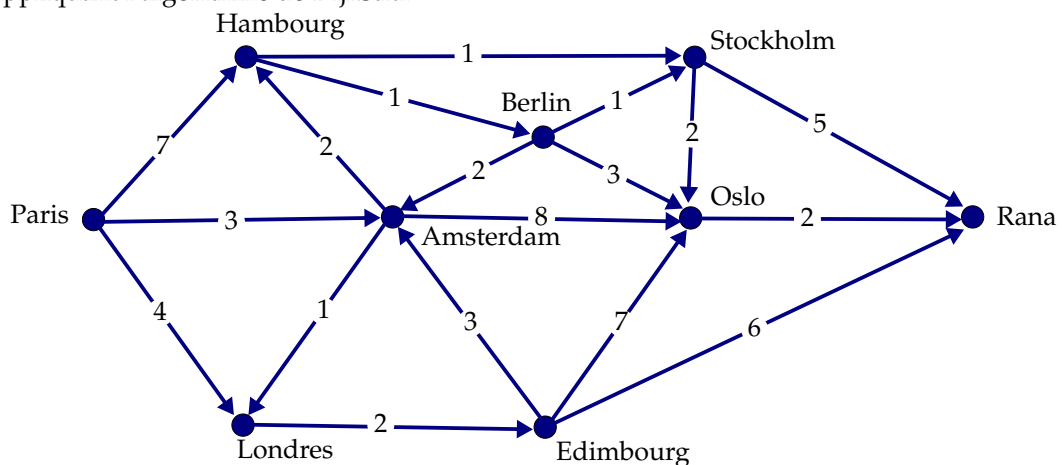
1. Appliquez manuellement l'algorithme de Kruskal pour trouver l'arbre à construire et donner son coût.
2. Quel est l'inconvénient majeur du choix d'une telle structure pour la banque ?

Answer of exercise 1

1. ab, bc, cd, de, ef, fg : coût 31
2. Réseau non fiable, 1 coupure $\rightarrow$ perte de connexité

Exercice 2 Chemins

Un étudiant de l'Ensta souhaite se rendre à Rana. Après avoir fait le tour de quelques compagnies, il a recensé plusieurs connexions aériennes possibles lui permettant d'aller de Paris à Rana. Il les a représentées à l'aide du graphe suivant. Les valuations portées sur les arcs correspondent au temps nécessaire (en heures) pour parcourir ces arcs, compte tenu des éventuels temps de transit pour les escales.



Answer of exercise 2

Chemin de valeur 10 : P A H S O R

Exercice 3 Implémentation de l'algorithme de Kruskal

1. Comment exécuter et modifier un fichier python ?

a) Ouvrir un terminal

Touche windows > Écrire "terminal" > Entrée

b) Se déplacer dans le répertoire contenant le fichier `kruskal.py` en utilisant dans le terminal la commande `cd` (exemple : `cd TP1-arbres-chemins/sources`).

c) Pour exécuter le programme, tapez la commande suivante dans le terminal :

`python kruskal.py`

d) Ouvrir l'éditeur de texte `gedit` :

Touche windows > Écrire "gedit" > Entrée

e) Ouvrir dans `gedit` le fichier `kruskal.py`.

2. Compléter la méthode `def kruskal(g)` du fichier `kruskal.py` permettant d'appliquer l'algorithme de Kruskal à un graphe `g`.

Indication : Vous pourrez vous servir de la méthode `Graph.createACycle(Edge)` qui renvoie vrai si l'ajout d'une arête à un graphe crée un cycle et faux sinon.

3. Lancer le programme et vérifiez que vous obtenez la même solution qu'à l'exercice précédent.
4. Utiliser cette méthode pour résoudre le problème d'arbre couvrant de poids minimal sur les deux graphes représentés en Figure 1.
5. On souhaite maintenant pouvoir obtenir des arbres couvrants de poids maximal. Ajouter un argument booléen `computeMin` à la méthode `kruskal` indiquant si l'algorithme cherche l'arbre couvrant de poids minimal ou maximal. Adapter votre code afin de prendre en compte ce paramètre et calculer les arbres couvrants de poids maximal des graphes représentés en Figure 1.
6. (plus difficile, à faire après l'exercice d'implémentation de Dijkstra) La complexité de la méthode `createACycle` est mauvaise. Une façon plus efficace de tester si l'ajout d'une arête crée un cycle est de stocker pour chaque sommet un numéro associé à sa composante connexe. Ainsi, il sera simple de vérifier si l'ajout d'une arête $[ab]$ crée un cycle en testant si l'identifiant des composantes connexes de a et b est le même.
 - a) Créer la méthode `kruskalCC(Graph g)`.
 - b) Dans cette méthode créer un tableau d'entiers nommé `component` de taille `g.n` indiquant la composante connexe de chaque sommet. Initialement, `component[i]` est égal à `i` car tous les sommets sont dans des composantes connexes différentes.
 - c) Implémenter l'algorithme de Kruskal dans cette méthode `kruskalCC` en utilisant le tableau `component` pour tester la création de cycle.

Answer of exercise 3

Code Kruskal

```
int edgeIndex = 0;

/* While a spanning tree is not obtained and all the edges of the graph
have not been considered */
while (addedEdges < tree.n - 1 && edgeIndex < edges.size() - 1) {

    Edge currentEdge = edges.get(edgeIndex);

    if (!tree.createACycle(currentEdge)) {
        tree.addEdge(currentEdge);
        addedEdges++;
    }

    edgeIndex++;
}

return addedEdges < g.nodes.length - 1 ? null : tree;
```

Arbres couvrants minimaux - Figure 1

graphe 1 : (fe), (cd), (eg), (gc), (gh), (eb), (fa) de valeur 27

graphe 2 : (BF), (CD), (AC), (FE), (FD) de valeur 13

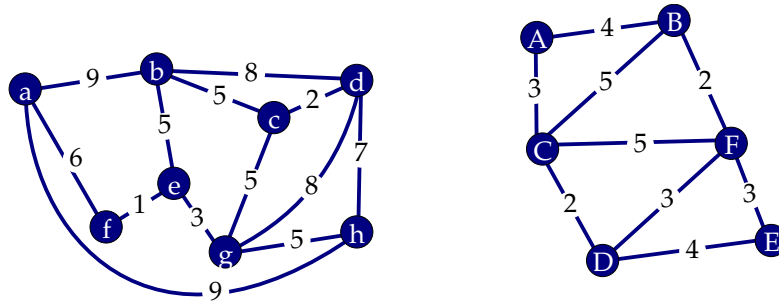


FIGURE 1 – Graphes pour lesquels on cherche un arbre couvrant de poids minimal.

Arbres couvrants maximaux - Figure 1

graphe 1 : (ab), (ah), (bd), (dg), (af), (be), (bc), de valeur 50

graphe 2 : (BC), (CF), (AB), (DE), (FD) de valeur 21

Code Kruskal Component

```
public static Graph kruskalComponents(Graph g) {

    Graph tree = new Graph(g.nodes);
    int addedEdges = 0;

    List<Edge> edges = g.getEdges();
    Collections.sort(edges);

    int[] component = new int[g.n];

    for(int i = 0; i < g.n; i++)
        component[i] = i;

    int edgeIndex = 0;

    /* While a spanning tree is not obtained and all the edges of the graph
    have not been considered */
    while(addedEdges < tree.n - 1 && edgeIndex < edges.size() - 1) {

        Edge currentEdge = edges.get(edgeIndex);

        /* If the nodes are in different components */
        if(component[currentEdge.id1] != component[currentEdge.id2]) {
            tree.addEdge(currentEdge);
            addedEdges++;

            for(int i = 0; i < g.n; i++)
                if(component[i] == component[currentEdge.id2])
                    component[i] = component[currentEdge.id1];
        }

        edgeIndex++;
    }

    return addedEdges < g.nodes.length - 1? null: tree;
}
```

Exercice 4 Implémentation de l'algorithme de Dijkstra

1. Compléter la méthode `def dijkstra(g, origin)` du fichier `dijkstra.py` permettant d'appliquer l'algorithme de Dijkstra pour trouver, dans le graphe g , les plus courts chemins entre le sommet `origin` et les autres.
2. Vérifiez que vous obtenez une solution de même coût que celle obtenu dans l'exercice précédent.
3. Utiliser cette méthode pour résoudre le problème de plus courts chemins sur les deux graphes

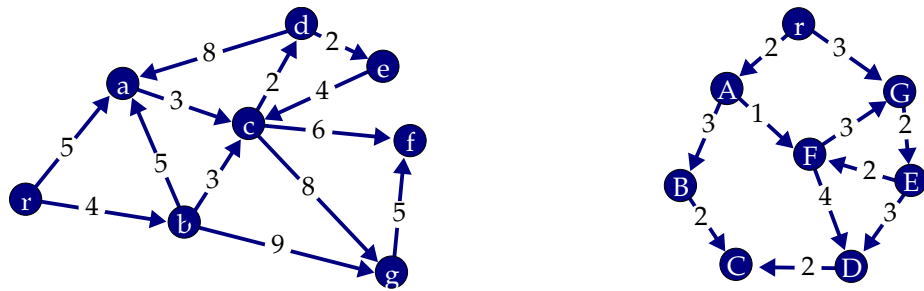


FIGURE 2 – Graphes pour lesquels on cherche le plus court chemin entre les sommets r et les autres sommets du graphe.

représentés en Figure 2.

Answer of exercise 4

Code Dijkstra

```
for(int j = 1; j < g.n; j++) {
    for(int y = 0; y < g.n; y++)
        if(!V2.contains(y) && g.adjacency[pivot][y] != 0.0)
            if(pi[pivot] + g.adjacency[pivot][y] < pi[y]) {
                pi[y] = pi[pivot] + g.adjacency[pivot][y];
                pred[y] = pivot;
            }

    pivot = -1;

    for(int z = 0; z < g.n; z++)
        if(!V2.contains(z) && (pivot == -1 || pi[z] < pi[pivot]))
            pivot = z;

    V2.add(pivot);
}

Graph tree = new Graph(g.nodes);

for(int i = 0; i < g.n; i++)
    if(i != r)
        tree.addArc(pred[i], i, g.adjacency[pred[i]][i]);

return tree;
```

Plus courts chemins - Graphe 1, Figure 2

ra=5
rb=4
rc=7
rd=9
re=11
rf=13
rg=13

Plus courts chemins - Graphe 2, Figure 2

rA=2
rB=5
rC=7
rD=8
rE=5
rF=5
rG=3

Exercise 5 Mise en niveau et Ordonnancement

On veut évaluer la durée minimale d'un gros projet de construction d'un barrage. On a évalué 9 tâches principales :

- construction des routes d'accès
- préparation des carrières et terrassements
- construction d'une cité pour les personnels
- commande du matériel hydraulique
- construction de la centrale

- f. construction du barrage et des digues
- g. construction des galeries et conduites forcées
- h. montage des machines
- i. essais de fonctionnement

Pour chaque tâche on a déterminé sa durée et les tâches qui doivent impérativement être terminées avant qu'elle ne commence. Ces informations sont résumées dans le tableau suivant. On a ajouté une tâche D (sans durée) correspondant au début du projet et une tâche F (sans durée) correspondant à la fin du projet.

Tâches	Durées (en mois)	Tâches préalables
D		
a	4	D
b	6	c
c	4	D
d	12	D
e	24	a,b
f	10	b
g	7	c
h	10	d,f,g
i	3	e, h
F		i

- On veut représenter le problème par un graphe : un sommet du graphe correspond à une tâche et il y a un arc de x vers y si la tâche x est une tâche préalable à y dans le tableau. Le graphe associé à un problème d'ordonnancement est sans circuit. Pourquoi? On va donc pouvoir dessiner ce graphe en "niveaux" (ou en ordre) : $N_0, N_1, \dots, N_i, \dots, N_n$. On place au niveau N_i toutes les tâches dont les prédécesseurs ont déjà été placés (sur les niveaux $N_0, \dots, N_{i-1}$). Dessiner le graphe obtenu (pour que le graphe soit clair, on place les tâches d'un même niveau l'une sous l'autre et dans l'ordre alphabétique).
- On éclate chaque sommet x ($x=a, \dots, i$) en un arc (x_1, x_2) et on value cet arc avec la durée de la tâche associée. Les arcs du graphe initial sont conservés et sont de valeur 0. Tracer le nouveau graphe obtenu de façon à conserver les informations précédentes.
- On veut trouver grâce à ce graphe la durée minimale du projet. Que doit-on chercher sur le graphe? Pensez-vous qu'on puisse utiliser l'algorithme de Dijkstra pour chercher cette durée? Il n'est pas demandé d'appliquer un algorithme pour résoudre le problème, mais on peut trouver la réponse facilement sur ce petit graphe.
- On appelle "tâche critique" une tâche dont la durée doit impérativement être respectée sous peine de retarder la durée totale du projet. Faire apparaître clairement sur le graphe les arcs qui correspondent à ces tâches critiques et les donner. Ce chemin est appelé "chemin critique".

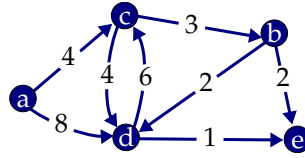
Answer of exercise 5

1.	Niveau	0	1	2	4	5	6
	Tâches	D	a, c, d	b, g	e, f	h	i
							F

-
-
- Pas possible d'utiliser Dijkstra car on cherche un chemin maximal. Solution : 37 mois
- Chemin critique : D, c, b, e, i, F

Exercise 6

On veut connaître les longueurs des chemins maximaux entre toutes les paires de sommets du graphe ci-joint. Quel algorithme allez-vous utiliser? Appliquez cet algorithme à l'exemple. Et si on voulait les chemins min?



Answer of exercise 6

— Aucun lien n'entre en $a \Rightarrow$ colonne $a = -\infty$

— Aucun lien ne sort de $e \Rightarrow$ ligne $e = -\infty$

Ils ne peuvent être des sommets intermédiaires, pas besoin de considérer $z \leftarrow a$ ou e .

	a	b	c	d	e
a	$-\infty$	$-\infty$	4,a	8, a	$-\infty$
b	$-\infty$	$-\infty$	$-\infty$	2,b	2,b
c	$-\infty$	3,c	$-\infty$	4,c	$-\infty$
d	$-\infty$	$-\infty$	-6,d	$-\infty$	1,d
e	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$

Liens direct

	a	b	c	d	e
a	$-\infty$	7,c	4,a	9,b	9,b
b	$-\infty$	$-\infty$	$-\infty$	2,b	2,b
c	$-\infty$	3,c	$-\infty$	5,b	5,b
d	$-\infty$	-3,c	-6,d	-1,b	1,d
e	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$

$z \leftarrow c$, liens de distance ≤ 3

	a	b	c	d	e
a	$-\infty$	$-\infty$	4,a	8, a	$-\infty$
b	$-\infty$	$-\infty$	$-\infty$	2,b	2,b
c	$-\infty$	3,c	$-\infty$	5,b	5,b
d	$-\infty$	$-\infty$	-6,d	$-\infty$	1,d
e	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$

$z \leftarrow b$, lien directs ou passant par b

	a	b	c	d	e
a	$-\infty$	7,c	4,a	9,b	10,d
b	$-\infty$	-1,c	-4,d	2,b	3,d
c	$-\infty$	3,c	-1,d	5,b	6,d
d	$-\infty$	-3,b	-6,d	-1,b	1,d
e	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$

$z \leftarrow d$, liens de distance ≤ 4

On reconstitue les chemins en regardant les prédécesseurs exemple, de a à e : a, c, b, d, e

Pour le cas des chemins minimums, on a un circuit absorbant entre c et d .